

AdaptiveCache: A Lifetime-Cost Study of Context Management for LLM Agents under Prefix-Cache-Aware Pricing

Vlad Cainamisir

Harvard John A. Paulson School of Engineering and Applied Sciences

CS2640: Modern Storage Systems — Final Report, May 2026

Abstract

LLM agents accumulate tens of thousands of tokens across long tool-use trajectories. Modern serving systems exploit prefix caching, billing already-computed tokens at roughly 1/10 the cost of uncached input. Compaction policies that drop “stale” context therefore look like obvious wins. We test some training-free versions of this hypothesis empirically. Across **eight heuristic compaction policies**, **~30 distinct task instances** (SWE-bench Lite live, τ -bench retail with chain extension), and **two model classes** (Qwen3-30B-A3B local, Anthropic Haiku 4.5 API), **no policy we tested Pareto-dominates none** (no compaction). The cause is a *cliff tax*: every compaction event mutates the prompt prefix, invalidates all downstream chain-hashed cache blocks, and re-bills the suffix at the uncached rate on the next call. We derive that compaction needs ~ 45 remaining steps to break even per compaction event; most agent trajectories from our chosen benchmarks have far fewer. We then present the constructive direction the negative result motivates: a system that captures observation KV to CPU on compaction, restores it on recall, and rotates the suffix’s already-cached K vectors in place by Δ tokens via the rotary composition identity $R(p + \Delta) = R(\Delta) \circ R(p)$, combined with an append-only auxiliary chain-hash registration that makes the cached blocks reachable under the recall-prompt’s chain hash. The mechanism is end-to-end operational on an H100 with the rotation correctness verified to bf16 cosine similarity > 0.999 across layers.

1 Introduction

Long-running LLM agents—ReAct [15], OpenHands, Claude Code—accumulate context across hundreds of tool calls. Every tool observation is appended to the conversation; the entire conversation is re-sent to the inference server every step. Without compaction the prompt grows unboundedly, eventually exceeding model context length and inflating per-step cost.

The prevailing assumption is that compaction is good for cost. The argument is intuitive: tool observations from

many steps ago are usually no longer needed; dropping them shortens the prompt. Many systems implement this—summarization at budget (OpenHands [1]), attention-based KV eviction [8, 14, 17], RL-distilled state replacement [16, 19], context folding [12].

The argument is incomplete. On modern serving systems, input tokens are billed at *two rates*: a cheap rate for tokens whose KV state is already cached server-side, and a $\sim 10\times$ more expensive rate for tokens whose KV state must be computed afresh. Cache reuse is keyed on byte-identical prefixes. Any compaction event modifies the prefix, invalidates downstream cache, and forces recomputation at the expensive rate.

This is the question the project asks: **when the cost of changing the prefix is $\sim 10\times$ the cost of not changing it, can a heuristic compaction policy still Pareto-dominate not compacting at all?**

We answer across multiple benchmarks, models, and policies. The answer, for our chosen benchmarks and models, is **no**. Given our limited budget, we were not able to run on long context benchmarks. On tasks under 100k tokens, compaction is a net negative.

Contributions. (1) A lifetime-cost evaluation framework that attributes per-step costs to compaction events across five provider price columns. (2) An empirical demonstration that no training-free heuristic compaction policy in our 8-policy survey beats *none* at the scale tested. (3) A back-of-envelope derivation of the cliff tax that explains *why*. (4) An inference-engine overlay that combines KV capture/restore with in-place rotary K-vector rotation and append-only auxiliary chain-hash registration—making the cached suffix reachable and *correct* under the recall-prompt’s chain hash, without re-prefilling. (5) An honest accounting of what we have shown (mechanism) and what we have not (Pareto win on a real workload).

Terminology. A *tool observation* (or just *observation*) is the text returned by an agent’s tool call (e.g. the contents of a `read_file`), appended to the conversation. *Compaction* is any operation that shortens the agent’s prompt by removing,

summarizing, or substituting an older observation. The *prefix* is the front of the request prompt and the *suffix* is its tail; cache reuse depends on prefix bytes being identical across requests. The *KV cache* is the set of per-token Key/Value attention activations the inference server keeps to avoid re-running the model forward on already-seen tokens. *Recall* is the inverse of compaction: re-introducing a previously compacted observation back into the prompt. A policy *Pareto-dominates* none on a workload if it resolves at least as many tasks at strictly lower cost.

2 Background

2.1 Prefix caching and the storage analogy

Modern inference servers (vLLM [7], SGLang [18], Anthropic, OpenAI) implement prefix caching: blocks of KV state are keyed by a chained content hash $H_i = \text{hash}(H_{i-1}, \text{tokens}_i)$. Two consecutive requests sharing a byte-identical prefix reuse the cached blocks; the first divergent block triggers a cache miss and forces re-*prefill* of everything after.

Concretely, in a vLLM-style engine: the *scheduler* assigns each request to a pool of fixed-size *KV blocks* on the GPU; a per-request *block table* maps logical token positions to the physical block IDs that hold their K and V activations; and a global *prefix-cache hash table* maps chain hashes H_i to the physical blocks that contain their content. When a request arrives, the scheduler walks the request’s chain hashes against the prefix-cache hash table and reuses any matching blocks; only the first uncached position onward is prefilled.

This is the storage-systems pattern the course studies: a content-addressable cache with locality-driven reuse, where the *layout* of items (their byte position in the prompt) determines hit rate. *What* you keep matters; so does *where* it sits.

2.2 The original AdaptiveCache thesis and prior measurements

Existing eviction methods [3, 8, 9, 14, 17] optimize *what to keep*; AdaptiveCache would also optimize *where to put it*, pinning stable high-value items at fixed prefix positions and demoting volatile items to the suffix.

Trace-driven measurements informing the design. Before running any policy comparison we measured four properties of real agent trajectories on the Hermes Agent Reasoning Traces corpus. (i) Three candidate importance proxies—how often a span is referenced by later tokens, semantic similarity to the goal, and attention weight from later positions—are pairwise near-zero correlated, so no single proxy captures “which token matters.” (ii) Across Qwen3 0.6B–8B, the first 10% of token positions carry 500–1500× more aggregate attention mass than the rest (*position primacy*), motivating a stable

prefix zone. (iii) Tool observations are 77% of agent-loop tokens, and large observations receive proportional attention—size alone is the wrong eviction signal. (iv) On production Anthropic Haiku trajectories, each compaction event costs \$0.10–\$0.15 of new uncached input on the next call and needs ~5+ steps to amortize. We work the underlying arithmetic out in §6.

2.3 Related work

Table 1 compares AdaptiveCache to existing systems on three properties relevant to our cost model.

System	Train. free?	Layout aware?	Cache aware?
FIFO / LRU	Yes	No	No
H2O, SnapKV, ScissorHands, PyramidKV	Yes	No	No
StreamingLLM [14]	Yes	Partial	Partial
LLMLingua [4, 5]	Optional	No	No
ReSuM [13]	Yes	No	No
SUPO [10]	RL	No	No
MEM1 [19]	RL	No	No
MemAgent [16]	RL	No	No
Context Folding [12]	RL	No	Partial
MEMENTO [6]	SFT	No	Partial
Claude Code [2]	Yes	Partial	Yes
AdaptiveCache (proposed)	Yes	Yes	Yes

Table 1: AdaptiveCache relative to existing context-management systems. Claude Code is the only prior system with all three properties.

3 Lifetime-cost framework

We define lifetime cost per task as the sum of per-step input/output costs across the trajectory plus any summarizer-LLM costs incurred by compaction:

$$\text{lifetime_}\$(\tau) = \sum_t (u_t p_u + c_t p_c + o_t p_o + w_t p_w) + \sum_e s_e$$

u_t, c_t, o_t, w_t are uncached / cached / output / cache-write tokens at step t ; prices come from a per-provider price table; s_e is the cost of a compaction-time summarizer call. This metric captures the cost of *changing* the prefix, which dominates for any policy that fires.

3.1 Evaluation methodology

Each SWE-bench Lite task ships with a `base_commit` (the pre-fix state of the repository), a `test_patch` (the regression tests added alongside the maintainers’ fix), and two named test lists: `FAIL_TO_PASS` (tests that should pass after the bug fix) and `PASS_TO_PASS` (tests that should remain

passing). We resolve a trajectory against this same gold set. For each completed trajectory we (1) replay its `edit_file` calls onto a fresh git checkout at `base_commit`, (2) apply the `test_patch` on top, (3) provision a per-instance Python 3.9 venv (uv as the package manager), (4) run `pytest` on the `FAIL_TO_PASS` list, and (5) declare the trajectory resolved iff every `FAIL_TO_PASS` test passes and no `PASS_TO_PASS` test regresses.

3.2 How this differs from the standard harness

Three differences are worth flagging upfront.

No docker. The official harness containerizes each instance. We do not, because our evaluation runs inside Modal and Modal does not support docker-in-docker. We instead use uv-managed Python 3.9 venvs per instance (many SWE-bench Lite `base_commits` do not build under Python 3.12). All tasks in our task list (psf/requests, pallets/flask, pylint, pytest) provision and execute under this setup.

Test-patch collisions are unvalidatable. If the agent’s edits touch the same lines `test_patch` adds, the patch fails to apply and the trajectory cannot be validated. We report both the full-set count and the validatable-subset count (e.g., 5/10 and 5/7) and use the latter for cross-policy comparison.

Single seed at $N=10$. Our main run is single-seed. At this scale a single task’s outcome moves the resolve count by ten percentage points; $\$/resolved$ divides by an integer denominator and is correspondingly noisy. We use it only alongside total $\$$ and the validatable-subset resolve count, and do not draw causal claims from $\$/resolved$ differences that hinge on ≤ 1 task.

4 Compaction policies under test

We implemented eight training-free compaction policies covering the natural design space (Figure 1). Each policy fires when the rendered prompt exceeds a soft token budget; what differs is *which* observations are dropped and *what, if anything*, replaces them. Table 2 gives the one-line mechanism for each, alongside how the policy fared in our empirical run.

To our knowledge no prior compaction system uses tool-graph supersession; the `consumption_evict` family is novel and works mechanistically (it correctly tags consumed observations across all 60+ trajectories we inspected) but the cliff tax dominates its byte savings (§6).

4.1 Benchmarks

We tested on (a) **SWE-bench Lite**, a coding-agent benchmark where the agent must produce a patch that passes the maintainers’ regression tests; we use a no-docker reimplement of the harness in which the agent operates against a cloned repository through six tools (`list_files`, `read_file`, `search`,

`edit_file`, `run_tests`, `submit`). And (b) **τ -bench retail**, a customer-service multi-turn dialogue benchmark; we extend it with a multi-customer chain harness that bundles N consecutive customer tasks into one continuous agent session (`chain_size=N`).

An initial run on τ -bench’s airline domain showed it to be the wrong benchmark for this study: single-customer tasks fit naturally in 16K, prefix-cache hit rate is already $\sim 95\%$, and observations cap at 950 tokens. *There is nothing to compact*, and policies that trigger end up $6\times$ more expensive than none. We moved on.

5 Empirical results

5.1 Main result: no policy beats none

Table 3 and Figure 2 report the main negative result. The cheapest compaction variant costs $2.2\times$ more than none for the same resolve count. Cost decomposition (Figure 3) shows the cost killer is `input_uncached`; Table 4 gives the same view restricted to *four heavy SWE-bench Lite tasks* (a subset of the slate above), the regime where `prefix_preserving` actually fires often enough for the per-line-item dollars to be meaningful.

5.2 Placeholder text affects agent behavior

When the action-graph supersession policy compacts an observation it leaves a short text token sequence in the prompt where the observation used to be—the *placeholder*. The placeholder’s content is a free design parameter: the eviction signal (which observation to drop, when) is unchanged across variants. We compared three placeholder contents: *plain* (an evicted marker only, e.g. [evicted: $\sim 5K$ tokens, consumed by `edit_file('skipping.py')`]); *facts* (the same marker plus a one-line list of the dropped file’s function names); and *outline* (the same marker plus line-numbered section headers and a “re-read to act” hint).

On the one validatable task where the four policies diverge, the *facts* variant performs worst: the agent fixates on the function names exposed by the placeholder and edit-reverts on the wrong function (18 edits, 0/2 `FAIL_TO_PASS`). The *plain* variant has no useful info to act on, forces a re-fetch, and resolves in 4 edits; the *outline* variant resolves in 3. The pattern is qualitatively consistent across two seeds, but it rests on a single task and we do not separate it from seed variance. The observation is that placeholder *content*—a knob orthogonal to the eviction signal—can change agent behavior; what to put in a placeholder is therefore part of compaction-policy design, not just bookkeeping.

The compaction design space — six approaches, one root question
 All 12 methods we tested answer the same question — "what should the agent's context look like after we touch it?" — but split on which signal they trust to decide.

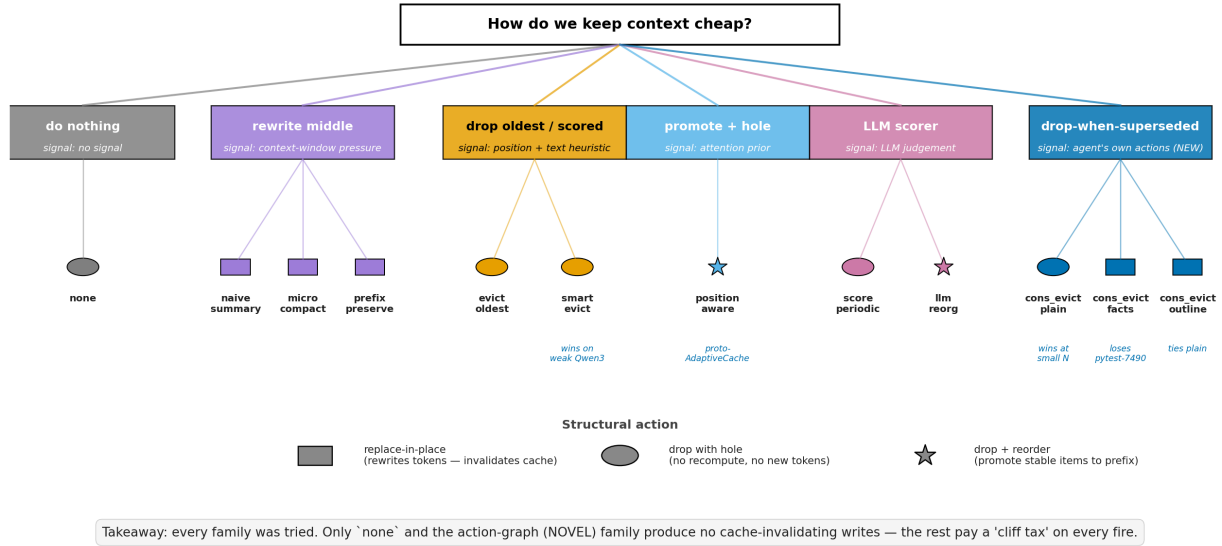


Figure 1: The eight training-free compaction policies tested. The rightmost branch (consumption_evict family) is novel: observations are tagged consumed by subsequent agent actions (e.g., a read followed by an edit on the same path consumes the read). Three placeholder-content variants are evaluated under the same eviction signal (§5.2).

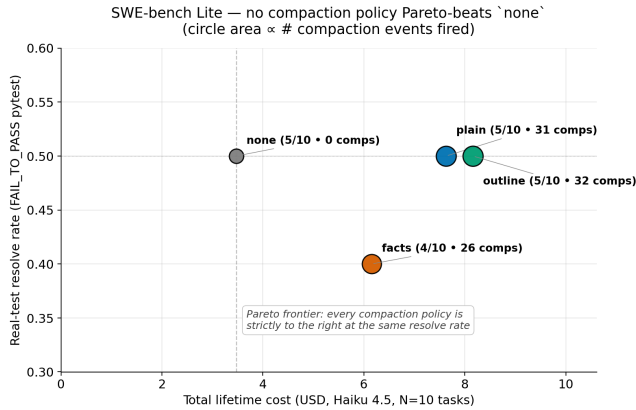


Figure 2: Pareto frontier (lower-left = better). none is alone on the frontier; every compaction variant is dominated.

5.3 Portability: rules are domain-specific

We extended τ -bench retail with a multi-customer chain harness. At chain_size=10 retail, action-graph supersession reached a max prompt of 36K (above the 29.75K trigger) but fired *zero* compactions, because retail tools (find_user_id_by_name_zip, get_order_details) do not match any of the SWE-bench-tuned supersession rules. The signal is correct; the rule set is domain-specific. The constructive direction is *boundary-aware* supersession: any

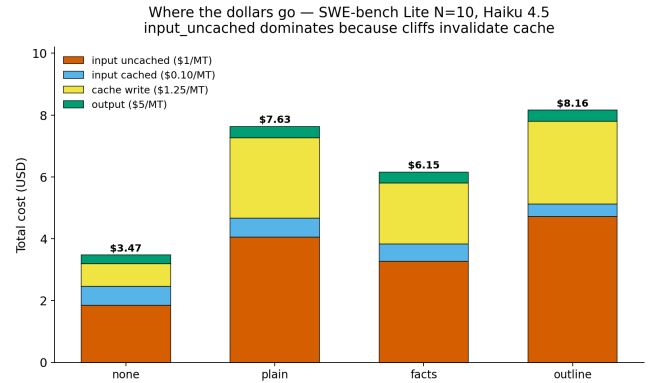


Figure 3: Cost decomposition. input_uncached dominates total cost on every compaction policy. The bytes saved on dropped spans show up as reduced future *cached* cost, which is already cheap.

subtask-end event consumes that subtask's tool observations.

6 Why no policy beats none: the cliff tax

The negative result of §5 has a single quantitative explanation.

Policy	Mechanism	Observed in our runs
none	Identity. No compaction.	Baseline; on the Pareto frontier.
evict_oldest (FIFO)	Drop the oldest tool observation when the prompt exceeds the soft budget.	Matches <code>none</code> resolve only with $2\text{--}3\times$ more steps; cost dominated by the cliff on each fire.
naive_summary	Concatenate all observations between the system prompt and the recent M turns; replace them with one summary message produced by a small LLM.	Summariser cost stacked on the cliff; consistently dominated.
microcompact	Rewrite each oversized observation in place to a 1–2 line summary; conversation structure preserved.	Same cost shape as <code>naive_summary</code> ; never on the frontier.
prefix_preserving	Freeze the first K turns as a stable prefix; replace the middle with one summary; keep the most recent M turns verbatim.	+200% cost vs <code>none</code> on the four heavy tasks (Table 4); 19 summariser calls + 19 cliffs.
position_aware	Drop oldest, but leave a fixed [evicted] placeholder in the message slot rather than removing it.	Same Pareto profile as FIFO; the placeholder does not save the cliff.
smart_evict (type-prior)	Rank eviction candidates by combining a per-tool-type prior (file reads > search results) with a textual-overlap reference count from later turns; drop the lowest-ranked.	+67% cost vs <code>none</code> at equal resolves on the four heavy tasks.
llm_reorganizer	A small LLM scores each tool observation 1–10 for likely future relevance; drop the K lowest-scored.	+39% cost vs <code>none</code> at equal resolves; the auxiliary LLM is the cheapest summariser-call profile we tried but still loses.
consumption_evict (action-graph supersedes)	Tag tool observations as <i>consumed</i> via tool-call semantics; drop only confirmed-stale observations. Five rules: <code>read_file</code> → <code>edit_file</code> on same path; <code>search</code> → <code>read_file</code> on same path; consecutive <code>run_tests</code> ; <code>edit_file</code> consuming prior reads of same file; <code>submit</code> consuming everything.	+120% cost vs <code>none</code> at equal resolves on the headline run (Table 3); the cost-cheapest compaction policy in our survey but still dominated.

Table 2: The eight training-free compaction policies under test. Each fires when the prompt exceeds a soft budget; the **Mechanism** column states what each policy does on a fire, the **Observed** column states how it fared empirically. None are on the Pareto frontier alongside `none`.

Policy	Real-resolved	Validatable	Total \$	\$/resolved	Compactions	Mean steps
none	5/10	5/7 (71%)	3.47	0.69	0	25.5
consumption_evict (plain)	5/10	5/7 (71%)	7.63	1.53	31	30.6
consumption_evict_facts	4/10	4/8 (50%)	6.15	1.54	26	32.5
consumption_evict_outline	5/10	5/7 (71%)	8.16	1.63	32	32.4

Table 3: Main result. Haiku 4.5, $N=10$ SWE-bench Lite, single seed, `FAIL_TO_PASS` validation. The cheapest compaction variant costs $2.2\times$ more than `none` for the same resolve count. **No policy in our survey Pareto-dominates `none` at this scale.**

Policy	u\$	c\$	w\$	tot\$
none	1.155	0.334	0.000	1.611
llm_reorganizer	1.646	0.293	0.163	2.244
smart_evict	1.954	0.468	0.088	2.686
prefix_preserving	3.685	0.329	0.071	4.842

Table 4: Cost decomposition (4 heavy SWE-bench Lite tasks). `u$=input_uncached`, `c$=input_cached`, `w$=cache_write`. `prefix_preserving`’s 19 cliffs generate \$3.69 of new uncached input vs `none`’s \$1.16.

6.1 Pricing

Every modern provider bills input tokens at two rates. On Anthropic Haiku 4.5: cached input at $\sim\$0.10/\text{MTok}$, uncached at

$\sim\$1.00/\text{MTok}$ —a $10\times$ ratio. *Provider-invariant*: holds across Sonnet, OpenAI gpt-4.1, gpt-4.1-mini, and Qwen self-host columns.

6.2 What a compaction event does

The serving system keys cached blocks by chained hashes $H_i = \text{hash}(H_{i-1}, \text{tokens}_i)$. Replacing a 5K-token observation at byte position 5,000 with a 50-token placeholder changes H_i at that position—and therefore every H_j for $j > i$. On the next request, the cache walk stops at position 5,000; everything after is recomputed and re-billed at the uncached rate. Figure 4 visualizes the mechanism, and the fix our system applies.

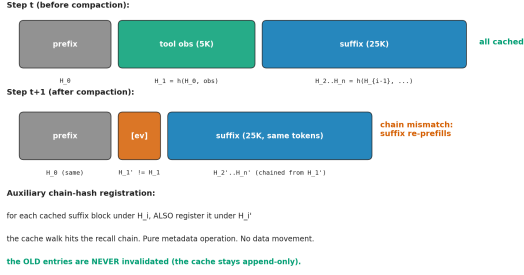


Figure 4: The chain-hash invalidation problem. The placeholder changes $H_1 \rightarrow H_1'$, which changes every downstream H_j' . Same suffix tokens, different cache key. Auxiliary chain-hash registration (bottom) restores the cache hit by adding a second key for the same physical blocks, append-only.

6.3 Break-even arithmetic

Let B_{save} be bytes saved per future step, B_{rebill} be bytes from the compaction position to the prompt tail at compaction time, $r = p_u/p_c \approx 10$, and N be remaining steps after the compaction event.

Compaction saves $N \cdot B_{\text{save}} \cdot p_c$ over the remaining trajectory. It costs $B_{\text{rebill}} \cdot p_c(r-1)$ on the next call. Break-even:

$$N \geq (r-1) \cdot \frac{B_{\text{rebill}}}{B_{\text{save}}} = 9 \cdot \frac{B_{\text{rebill}}}{B_{\text{save}}}$$

Table 5 works this through with concrete numbers under Anthropic Haiku 4.5 pricing. For a typical compaction event ($B_{\text{save}} \approx 5,000$, $B_{\text{rebill}} \approx 25,000$ at mid-trajectory), **compaction needs ≥ 45 remaining steps to break even**. The trajectories in our $N=10$ run average ~ 26 steps total.

Quantity	Value
Prompt size at compaction	30,000 tokens
Tool observation being compacted	5,000 tokens
Placeholder size	~ 50 tokens
Suffix re-billed (uncached, next call)	25,000 tokens
Next-call cost, cached	\$0.0025
Next-call cost, uncached	\$0.0250
Δ on next call	\$0.0225
Per-step savings (5K cached @ \$0.10/MTok)	\$0.0005 / step
Break-even (Δ / savings)	45 steps

Table 5: Worked example for the break-even arithmetic at Anthropic Haiku 4.5 pricing. A mid-trajectory compaction event needs ~ 45 future steps to recoup its cliff cost; agent trajectories at the scale we tested average ~ 26 steps.

Compaction events usually happen mid- to late-trajectory, leaving 5–15 amortization steps. **The cliff is structurally unrecoverable on most realistic agent trajectories.** Compaction *as a cost lever* does not beat none; compaction *as a*

behavior-modification lever sometimes does (clearing working memory at the right moment can change the agent’s next action), but that is not a cost story and is not what the compaction literature claims.

7 A cliff-free compaction substrate

Re-derive the break-even with $r = 1$ (no cliff penalty): $N \geq 0$, trivially satisfied. Any compaction is profitable from the first future step. The goal is therefore simple: **when the agent compacts an observation, the engine should keep the suffix’s KV state usable; when the agent later needs the observation back, putting it back should not require any model recomputation.**

Foundations. Our system builds heavily on the Memento overlay [6], which introduced the idea of representing compacted observations as inline marker tokens that the inference engine can interpret as KV-block boundaries without retraining the model or modifying its tokenizer. Memento’s mechanism handles the compaction direction; we extend it with bidirectional recall—KV capture and restore, plus the two suffix repairs described below—so that a recalled observation costs essentially nothing beyond a memory copy.

Figure 5 shows what this looks like for a single observation across three moments in time. At t_0 everything is normal: the observation is in the prompt, its KV state lives in the engine’s paged cache. At t_1 the policy compacts: the prompt now carries a small placeholder, and the engine has copied the observation’s KV to a CPU side cache; the GPU blocks return to the engine’s LRU pool. At t_2 the policy recalls: the engine copies the saved KV back to fresh GPU blocks at the observation’s original logical position. No recomputation; only memory movement.

The hard part is making the *suffix*—the part of the prompt written *during* the compacted period—still work after this splice. Two things go wrong with a naive splice. First, the suffix’s K vectors carry positional encoding for the positions they had during the compacted period, not the positions they now occupy after the observation has been restored. Second, the prefix-cache hash table indexes those blocks under a chain hash that runs through the placeholder, so the recall prompt’s cache walk misses. We fix the first by rotating the cached K vectors in place by Δ tokens (a per-layer tensor op that reuses the engine’s rotary kernel and adds no model forward pass). We fix the second by adding a second key for the same physical blocks to the prefix-cache hash table—append-only, never invalidating the original.

Figure 6 shows where these operations live in the system. The agent loop and compaction policy run in the main process; the scheduler and GPU worker run in the inference-engine subprocess. Four cross-process JSONL queues (KV capture, KV restore, K rotation, auxiliary chain-hash insert) carry the policy’s decisions over the IPC boundary; the engine drains

them on each scheduling tick.

7.1 Why text-level recall isn’t enough

The naive recall is text-level: substitute the original observation for its placeholder in the prompt and let the engine re-*prefill*. Two variants—in-place (at the original position) and append (at the prompt tail)—differ in how much of the suffix the re-*prefill* catches. Figure 7 reports cache hit rates from an evaluation in which the engine’s normal LRU eviction is allowed to release blocks (rather than *refcount*-pinning the observation’s blocks): in-place recall eats a 27-point cache penalty relative to a no-recall control, even though the suffix tokens themselves are unchanged. The penalty is purely a chain-hash mismatch: the suffix’s parent chain runs through the original observation in the recall prompt but through the placeholder in the cache. The fix is engineering the cache to hold both keys.

7.2 KV-level mechanisms in the engine

The full mechanism (Figure 5, $t_1 \rightarrow t_2$) combines three engine-side operations on top of a side cache.

Side cache: capture and restore. At t_1 , the engine copies the observation’s K/V from GPU to CPU pinned memory and indexes it by a content hash; the GPU blocks are released without a *refcount* pin and fall to the engine’s LRU free queue. At t_2 , the scheduler allocates fresh GPU blocks via the engine’s allocator, queues a CPU→GPU memcpy, and splices the new blocks into the request’s block table at the original logical position. The splice must *replace* the affected slice of the block-table list; an insert grows the table and overshoots `num_computed_tokens` on a later step, tripping a scheduler assert.

In-place rotary K rotation. Modern LLMs (Qwen3 included) encode token position via rotary positional embedding [11] (RoPE): each token’s K vector is rotated by an angle proportional to its absolute position, with the rotation applied at compute time and “baked” into the stored K. After the splice above, the suffix’s K vectors carry the angles they had during the compacted period, but their logical positions have shifted by $\Delta = m_{\text{obs}} - p_{\text{placeholder}}$ tokens (the obs’s $\sim 5\text{K}$ tokens replace the placeholder’s ~ 50). For $\Delta \approx 5000$, high-frequency rotary channels wrap many times around the unit circle and suffix attention becomes random.

Rotary embeddings compose: $R(p + \Delta) = R(\Delta) \circ R(p)$. Multiplying the cached suffix K vectors by $R(\Delta)$ in place corrects their phase without any model forward pass. The operation is per-layer, fixed-shape, and reuses the engine’s shipped rotary kernel. We verified composition with three microbenchmarks: in fp32, cosine similarity **1.0000000** across all (p, Δ) pairs tested; the engine kernel composes the same way; in bf16 in the paged KV layout, cosine similarity exceeds 0.999 across layers and V tensors are bit-identical pre/post.

Auxiliary chain-hash registration. For each cached suffix block under the compacted-period chain hash, the scheduler *also* inserts it into the prefix-cache’s hash table under the recall-prompt’s chain hash. Pure metadata operation—no data movement, no kernel work, append-only. The *old* entries are never invalidated, a project-wide invariant: future requests that walk the old chain still find the same physical blocks. The storage-systems insight is that the engine’s prefix cache is content-addressable and position-immutable by default; we make it position-mobile by **adding** new keys, not removing old ones.

End-to-end test. On a single SWE-bench Lite task (Qwen3-30B, H100), the engine stats record three captures, four restores, and two rotations covering 1260 blocks. The salient number is **zero refcount pins applied**: the captured GPU blocks were not pinned, the request truncation released them under LRU pressure, and the recall still recovered correct KV state. Block-level release is directly observed in a parallel memory trace—the scheduler’s free-block count oscillates across capture / restore / rotate events as expected.

Engine-side capacity returned. A complementary measurement: under `-no-pin`, captured observation blocks fall to the engine’s LRU free queue and become reusable by other concurrent requests on the same engine. On a 14-step `lru-kvrestore` trajectory of `pytest-7490` (Qwen3-30B-A3B, bf16, GQA with 4 KV heads, 16-token blocks at 1.50 MB each), the GPU memory trace records a peak instantaneous free-block delta of 988 blocks ($\approx 1.55\text{ GB}$) and a cumulative 1973 blocks ($\approx 3.10\text{ GB}$) recycled into the free pool across 12 capture events. The mechanism’s own GPU work (capture / restore / rotate) totals 1.78 s out of 87.3 s of engine wall ($\approx 2\%$), so this capacity return is essentially free. We do *not* claim memory release to the OS: PyTorch’s caching allocator never shrinks (`alloc_bytes` stayed constant at 73.2 GB), so other Linux tenants still see the GPU as fully occupied. The win is internal to the engine. In a multi-agent serving topology where one engine serves N concurrent agents, every $\approx 1.5\text{ GB}$ returned per agent trajectory is one additional concurrent working set fitting in the same hardware budget; the gain scales linearly with compaction events per trajectory.

7.3 Alternative: attention-mask eviction

An alternative to moving KV bytes is to leave the observation’s blocks pinned in their GPU positions and toggle attention *off* during the compacted period via the request’s masked-block list; recall becomes a mask flip. In mechanism-internal comparisons, mask-based eviction wins by -55% wall on the regime with the most compaction events and loses on light-compaction regimes where the marker tax dominates; on a four-task four-seed recall-heavy evaluation, attention-unmask recall beats text-append recall by -11% wall mean.

Lifecycle of a tool observation under KV-pointer recall.

The observation's KV state is captured to a side cache on compaction (t_1) and restored to its original logical positions on recall (t_2); the prefix cache is preserved across both transitions.

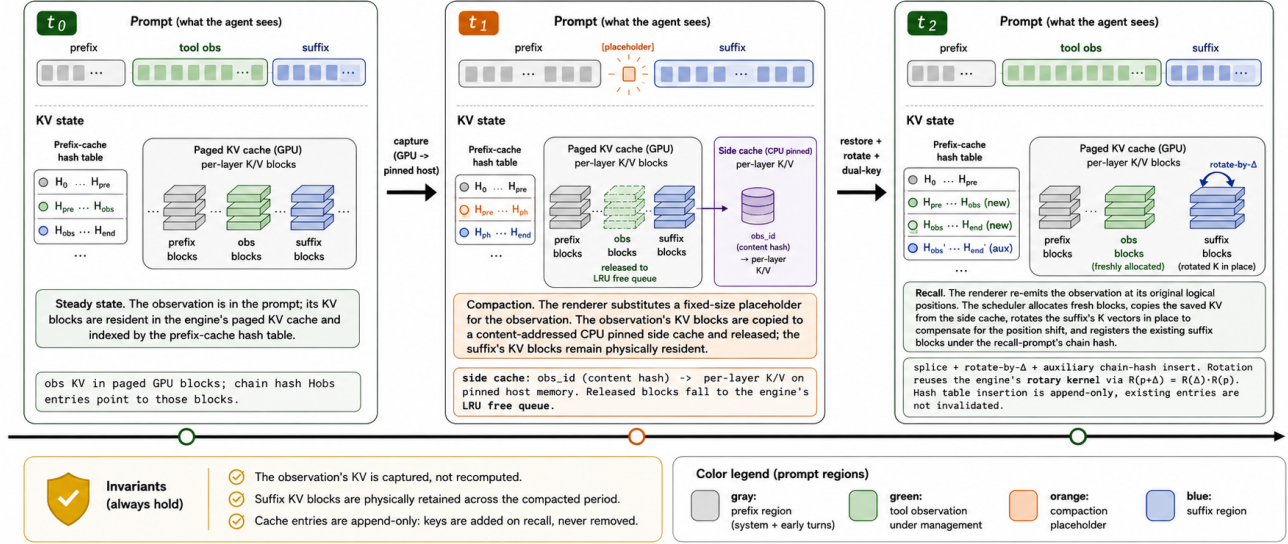


Figure 5: Lifecycle of a tool observation under KV-pointer recall. t_0 (steady state): the observation is in the prompt and its per-layer K/V blocks are resident in the engine's paged KV cache, indexed by the prefix-cache hash table. t_1 (compaction): the renderer substitutes a fixed-size placeholder for the observation; the engine captures the observation's K/V to a content-addressed CPU pinned side cache and releases the GPU blocks; the suffix's blocks remain physically resident but become reachable only under a compacted-period chain hash. t_2 (recall): the engine allocates fresh blocks, restores the saved K/V from the side cache, rotates the suffix's K vectors in place by Δ tokens to compensate for the position shift, and registers the existing suffix blocks under the recall-prompt's chain hash. Three invariants hold across the lifecycle: the observation's K/V is captured (not recomputed), the suffix blocks are physically retained, and prefix-cache entries are append-only.

The trade-off is structural: mask-based trades GPU bytes for engine simplicity. Neither is a comparison to none.

7.4 Engineering effort

Getting the mechanism end-to-end took three days of continuous engineering and a sizeable patch set against vLLM. The final delta is **+2225 / -44 lines across 12 files in v1.1m/v1/** on top of a pinned base commit, landed across 33 commits. The bulk of the change is in two files: the scheduler (`sched/scheduler.py`, 19 hunks containing most of the capture / restore / rotate / dual-key control flow) and the GPU worker (`worker/gpu_model_runner.py`, +461 lines covering the K-rotation kernel call, the splice execution path, and the GPU memory trace). The remainder is split between the single-type KV-cache manager (capture / restore / rotate plumbing), the block-masking overlay (`core/block_masking/` — `memento_store`, `tracker`, `package init`), and smaller hunks in the block pool, the KV-cache manager, the scheduler-output struct, the request struct, and configuration plumbing. Two patches were reverted in-session because they would have violated the project-wide “never invalidate” invariant (§7).

The recurring source of difficulty is structural rather than algorithmic: the system has *four* cross-process state surfaces (Figure 6) and *two* tokenizers, and any divergence between policy-side and engine-side views manifests as either a silent recall miss or a hard scheduler assert. Representative classes of issue: vLLM API drift between minor releases (pin to a specific commit, not a release tag); a tokenizer mismatch on the obs-id content hash that prevented recall from ever matching until both sides routed through the engine's tokenizer; a block-table splice that grew the table by inserting rather than replacing the affected slice, eventually overshooting `num_computed_tokens` on a later step; stale chain-hash entries under no-pin operation, patched with defensive clamps because the principled fix would have required invalidating cache entries.

8 What we have not shown

We are deliberate about distinguishing *mechanism progress* from *empirical Pareto-win against the baseline*. The KV-pointer recall mechanism is end-to-end operational; the Pareto-win against none on a real task list **has not yet been measured**. The project was compute-bound throughout; an

KV-pointer recall: agent loop + inference engine over IPC

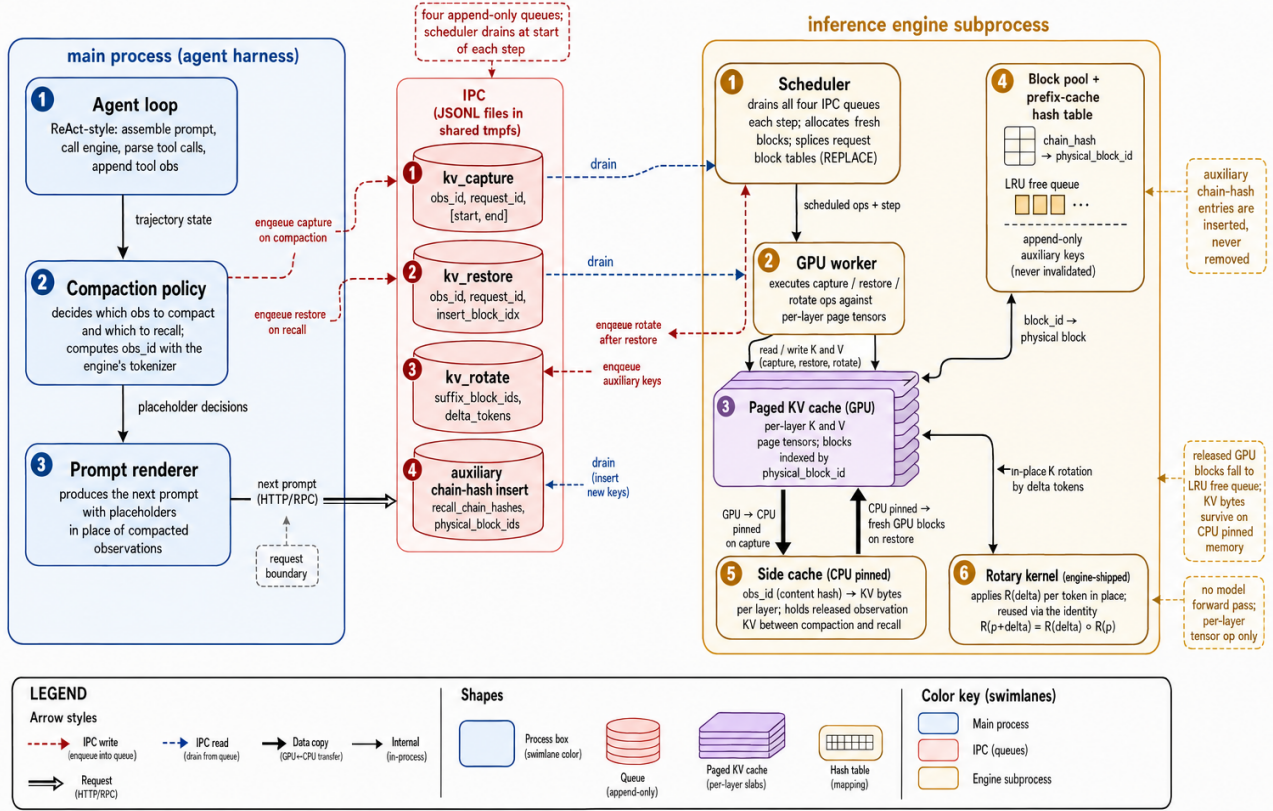


Figure 6: System architecture. The agent loop, the compaction policy, and the prompt renderer run in the main process; the scheduler and GPU worker run in the inference-engine subprocess. Coordination across the boundary is via four append-only JSON-line queues in shared tmpfs (KV capture, KV restore, K rotation, auxiliary chain-hash insert) drained by the scheduler on each step. A content-addressed side cache lives engine-side and holds CPU pinned K/V for released observations; the rotary kernel and prefix-cache hash table are reused unmodified from the engine. Hash determinism across the process boundary requires matching tokenizer and PRNG seed; hashing the wrong tokenizer was an early build bug.

end-to-end head-to-head run did not land within our budget.

Phase status at submission. A validation run that forces recall on every memento'd observation (pytest-7490, Qwen3-30B-A3B, two seeds, max_steps=20, recall_low_water=2.0, -no-pin) survives 12 successive captures and 12 successive recalls without crashing, returns ≈ 1.55 GB of GPU KV blocks to the engine's free pool at peak, and pays $\sim 2\%$ of engine wall on the splice/restore/rotate work itself. **The remaining honest gap is a +57% per-step wall over the no-compaction baseline on the same trajectory.** That gap is not the rotation mechanism; it is prefill cost on tokens that should have been cache hits. We traced it to chain-hash mutations introduced when the renderer transitions between mementoed and un-mementoed forms of a message. One transition (the first time an observation is mementoed) is now closed by a renderer change that always emits `<tool_response>...</tool_response>` markers, so with- and without-memento renderings differ only in

their appendix bytes. The other (un-mementoing on recall) remains open: the auxiliary chain-hash registration of §7 is the intended fix, the engine-side code is wired, but in our most recent run the trigger did not fire in production (dual_key_inserts=0). The fix in flight is to invoke the auxiliary insert on every recall regardless of rotation delta; this is not validated in this report.

- **No apples-to-apples mechanism vs. none.** Single-task end-to-end tests do not overlap with the small task list where we measured the none baseline. Vanilla baseline = 1/5 resolved on that list; a future list with a higher resolve ceiling is preferable.
- **Recurring scheduler assert.** `total_num_scheduled_tokens > 0` occasionally trips on long evaluations. Patched with defensive clamps; not fixed at root. The likely cause is the stale chain-hash entries described above.

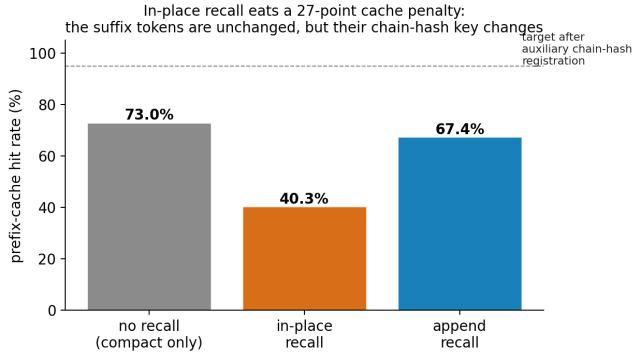


Figure 7: Cache hit rate across text-level recall variants. In-place recall loses 27 points to no-recall despite identical suffix tokens; the penalty is the chain-hash mismatch.

- **Multi-seed at $N=10$ for the main result.** The single-seed run gives the negative result; a multi-seed run is needed to bound the seed-driven per-task variance flagged in §5.2 and confirm the Pareto outcome.

9 Future work

The most direct next steps, all gated on compute rather than design:

- **Apples-to-apples mechanism vs. none.** A shared task list with multi-seed evaluation under both configurations. This is the missing empirical comparison the constructive direction needs.
- **Multi-seed $N=10$ main result.** 3 seeds $\times$ 4 policies $\times$ 10 tasks. Replaces the single-seed numbers with a paired test.
- **End-to-end validation of the auxiliary chain-hash registration** on a recall-heavy workload, looking for the predicted 40% $\rightarrow$ >90% in-place-recall cache-hit jump.
- **Boundary-aware supersession.** Add a rule that any subtask-end event consumes that subtask’s tool observations and re-run the multi-customer chain evaluation, where the SWE-bench-tuned rules currently fire zero compactions.
- **Long-context regimes.** The cliff-tax derivation predicts compaction’s penalty shrinks once trajectories are long enough to amortize the re-prefill; the inflection point is an empirical question we did not reach.
- **A Yellow-Pages-style benchmark.** SWE-bench Lite trajectories are largely linear—the agent reads, edits, and moves on, with limited revisits to earlier observations. A benchmark in which the agent must repeatedly look up information across many entries (think directory-style lookups, where an observation seen at step k is needed

again at step $k+40$) would exercise compaction *and* recall together at high frequency. This is exactly the regime where KV-pointer recall amortizes its overhead: each compaction-recall cycle reuses the cached suffix instead of re-prefilling it. We expect the cliff-free substrate to show its strengths here in a way SWE-bench Lite cannot.

The closest thing we have to a positive number for the constructive direction is the recall-heavy mechanism-internal comparison (−11% wall, mask vs append recall), which is a within-mechanism comparison and not a comparison to none.

10 Goal achievement

75% (baseline + simple pruning). Achieved. none, FIFO, and position-aware placeholder eviction implemented and measured across SWE-bench Lite live and τ -bench retail with real Anthropic cache-read accounting.

100% (AdaptiveCache compaction + reordering, full comparison). Eight training-free policies implemented and compared (§4); the comparison shows none beats none at the scale tested. Reordering as a static-prompt operation was *not* pursued: the cliff-tax analysis (§6) showed it could not pay off without changing the cost model itself.

Pivot. On finding that the cliff is structural, we redirected effort from heuristic scorers (the original 125% goal: an MLP scorer trained on attention oracle labels) to attacking the cliff directly. The result is the engine overlay of §7: KV capture/restore, in-place rotary rotation, and append-only auxiliary chain-hash registration. This is a different program from the original 125% but it subsumes the layout-reorganization goal: the auxiliary chain-hash registration is the storage-systems primitive that makes “re-position cached blocks without paying for it” actually possible.

11 Conclusion

We set out to test whether layout-aware, training-free heuristic compaction can Pareto-dominate no compaction on the lifetime cost of LLM agents. Across the policies, models, and benchmarks tested, the answer is no. The reason is the cliff tax: byte-level eviction structurally costs more on the next call than it saves over the remaining trajectory, on the agent workloads at the scale we evaluated.

Inside the negative result we report two findings worth keeping: an action-graph supersession signal that uses agent tool-call semantics to identify provably stale tool observations, and an observation that placeholder content is its own design knob, with two placeholder texts under the same eviction signal producing qualitatively different agent trajectories.

The constructive direction is to change the cost model itself—to make the cached suffix remain reachable and correct

even after the prompt prefix changes. We built an inference-engine overlay that does this through KV capture and restore, in-place rotary rotation of the suffix’s K vectors, and append-only auxiliary chain-hash registration. The mechanism is verified end-to-end on H100; the empirical demonstration that the resulting recall is genuinely cheaper than `none` is the missing comparison.

The takeaway: prefix-cache reuse is determined by chain-hash invariants, not by which bytes look stale. Any compaction policy that mutates the prompt prefix without preserving the cached suffix’s reachability under the new chain hash pays the cliff tax this report measures.

References

- [1] All-Hands.dev. OpenHands context condensation. <https://github.com/All-Hands-AI/OpenHands>, 2025.
- [2] Anthropic. Claude Code compaction source (`src/services/compact/`), 2025. Internal source, became publicly accessible 2026.
- [3] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, et al. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.
- [4] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.
- [5] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2024.
- [6] Vasilis Kontonis, Yuchen Zeng, Shivam Garg, Lingjiao Chen, Hao Tang, Ziyang Wang, Ahmed Awadallah, Eric Horvitz, John Langford, and Dimitris Papailiopoulos. MEMENTO: Teaching LLMs to manage their own context. *arXiv preprint*, 2026. Microsoft Research.
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PageAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [8] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- [9] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. ScissorHands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] Yucheng Lu et al. Scaling LLM multi-turn reinforcement learning with end-to-end summarization-based context management. *arXiv preprint*, 2025.
- [11] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.
- [12] Yuxiao Sun et al. Context folding: Active context management for LLM agents via learned branch-return mechanisms. *arXiv preprint*, 2025.
- [13] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, et al. ReSum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*, 2025.
- [14] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [16] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, et al. MemAgent: Reshaping long-context LLM with multi-conv RL-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.
- [17] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [18] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of

structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.

- [19] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025.